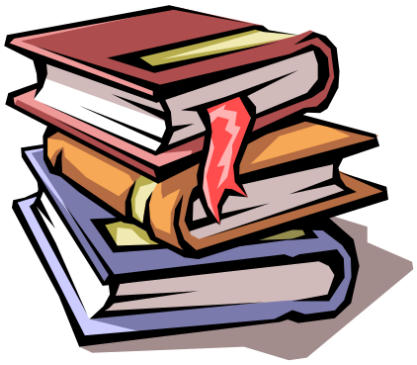
FPT UNIVERSITY

Lab Report

LAB-02 – VIẾT DEVICE DRIVER & QUẢN LÝ FILE SYSTEM

Embedded Linux Systems – ARMv7 Cortex-A9 (QEMU vexpress-a9)



Student Name	Phạm Võ Hoàng Nhân
Student ID	SE200778
Class	IC2008
Lecturer	Phạm Thế Vinh
Date	28/09/2026

1. Thông tin sinh viên và môi trường

Họ và tên	Phạm Võ Hoàng Nhân
MSSV / Lớp	SE200778 / IC2008
Ngày thực hiện	28/09/2026
Kernel target	Linux 5.15.0 (zImage từ LAB-01)
Vermagic	`5.15.0 SMP mod_unload ARMv7 p2v8`
Toolchain	`arm-linux-gnueabihf-gcc` 15.2
Máy host	Ubuntu 26.04 (phần C chạy trên host, dùng kernel của host)
Target	QEMU `vexpress-a9`, Cortex-A9

Phần A, B, D chạy trong QEMU trên kernel 5.15 build ở LAB-01. Phần C (MTD, JFFS2) chạy trực tiếp trên máy host Ubuntu với `sudo`, vì `nandsim` là module của kernel host.

2. Sơ đồ kiến trúc driver

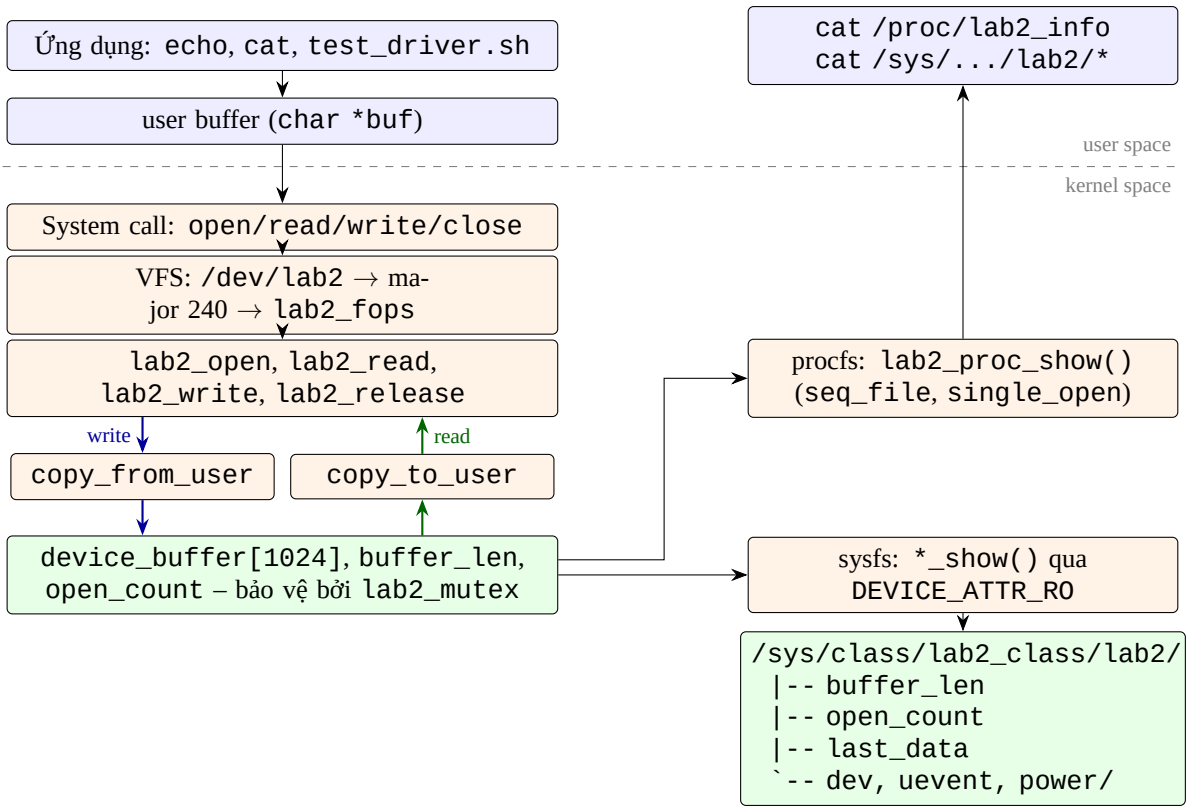


Figure 1: Kiến trúc lab2_driver: đường ghi (xanh dương) và đường đọc (xanh lá)

Khi user chạy `echo 'abc' > /dev/lab2`, shell gọi `open()` rồi `write()`. VFS nhìn inode của `/dev/lab2`, thấy đây là character device major 240, và chuyển lời gọi tới bảng `lab2_fops` mà driver đã đăng ký bằng `register_chrdev()`. `lab2_write()` khóa `lab2_mutex`, dùng `copy_from_user()` chép dữ liệu từ buffer của user vào `device_buffer`, cập nhật `buffer_len` rồi mở khóa. Chiều đọc đi ngược lại qua `copy_to_user()`. Hai giao diện procfs và sysfs không đi qua `/dev/lab2` mà đọc thẳng các biến trong driver mỗi khi có người `cat`: procfs gom tất cả vào một file `/proc/lab2_info`, còn sysfs tách mỗi giá trị thành một file thuộc tính dưới thư mục của device trong `/sys/class/lab2_class/lab2/`.

3. Log thực hành

Phần A – Kernel module character device

```
./lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=8eac38c0fbb501fc63408
86b854f02be8ab2b0fb, with debug_info, not stripped
filename:       /home/dyk/embedded_lab2/driver/./lab2_driver.ko
version:        1.0
description:    LAB-02: Simple Character Device Driver
author:         Embedded Linux Lab
license:        GPL
srcversion:     4E89036A2D2BA07B7DE24B7
depends:
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

Figure 2: `file` và `modinfo lab2_driver.ko`

Module là ELF 32-bit ARM, và vermagic `5.15.0 SMP mod_unload ARMv7 p2v8` khớp với kernel build ở LAB-01, nên `insmod` trong QEMU không bị từ chối. Module được nạp tự động lúc boot (xem phần D, dòng `lab2_driver: loaded, major=240`), và `lsmod` cho `lab2_driver 16384 0 - Live`.

```
/ # test_driver.sh
=== LAB-02: Device Driver Test ===
[1] Checking lab2_driver...
lab2_driver already loaded
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
crw-rw-rw-    1 root     0         240,   0 Sep 28 14:18 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=10)
/ # test_driver.sh
=== LAB-02: Device Driver Test ===
[1] Checking lab2_driver...
lab2_driver already loaded
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
crw-rw-rw-    1 root     0         240,   0 Sep 28 14:18 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=18)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=19)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
  Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: device opened (count=20)
lab2_driver: received 10 bytes: [Message_1
]
lab2_driver: device closed
lab2_driver: device opened (count=21)
lab2_driver: sent 10 bytes to user
Message_1
lab2_driver: device closed
lab2_driver: device opened (count=22)
lab2_driver: received 10 bytes: [Message_2
]
lab2_driver: device closed
lab2_driver: device opened (count=23)
lab2_driver: sent 10 bytes to user
Message_2
lab2_driver: device closed
lab2_driver: device opened (count=24)
```

Figure 3: `test_driver.sh`: kiểm tra module, tạo node, ghi và đọc `/dev/lab2`

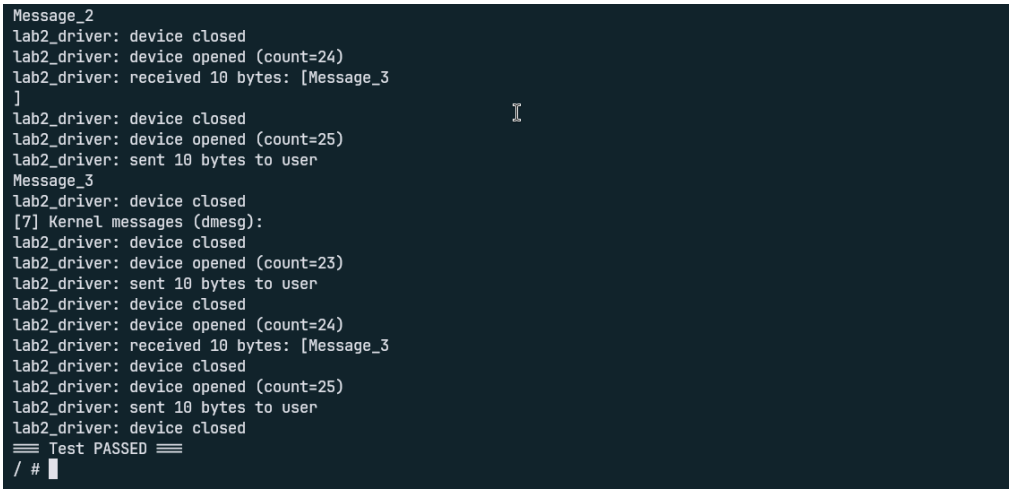


Figure 4: `test_driver.sh` (tiếp): ghi/đọc 3 lần và `dmesg`, kết thúc `Test PASSED`

Mỗi lần ghi đều thấy cặp log `device opened` → `received N bytes` → `device closed`, và lần đọc ngay sau trả lại đúng chuỗi vừa ghi (`Read back: [Hello from userspace, LAB-02!]`). Chuỗi 30 byte vì `echo` thêm ký tự xuống dòng ở cuối, nên trong log dấu `]` bị đẩy xuống dòng dưới. Ở ảnh đầu, lần chạy thứ nhất dừng sau `[4]`; em chạy lại thì script chạy hết. `open_count` tăng liên tục qua các lần chạy vì driver chỉ tăng trong `open` mà không giảm trong `release`, tức nó đếm tổng số lần mở chứ không phải số file đang mở.

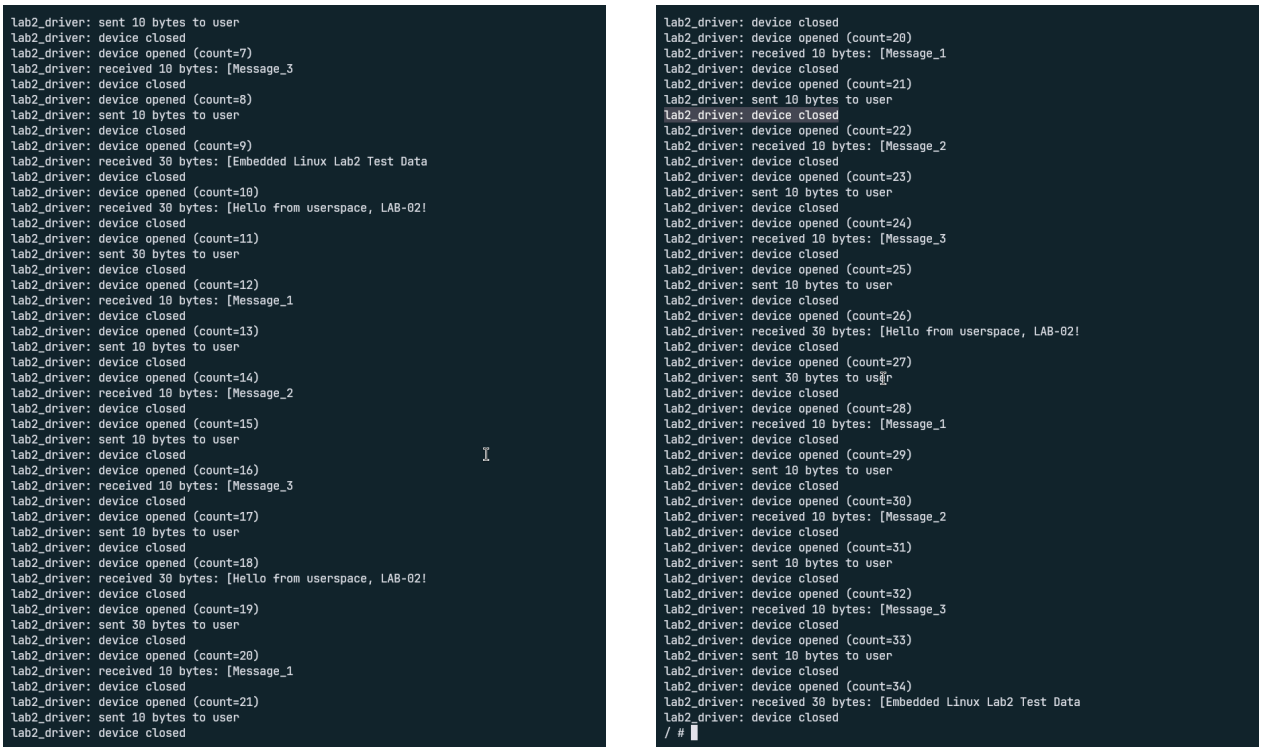


Figure 5: `dmesg | grep lab2`: toàn bộ thao tác từ lúc nạp module

Phần B – procfs và sysfs

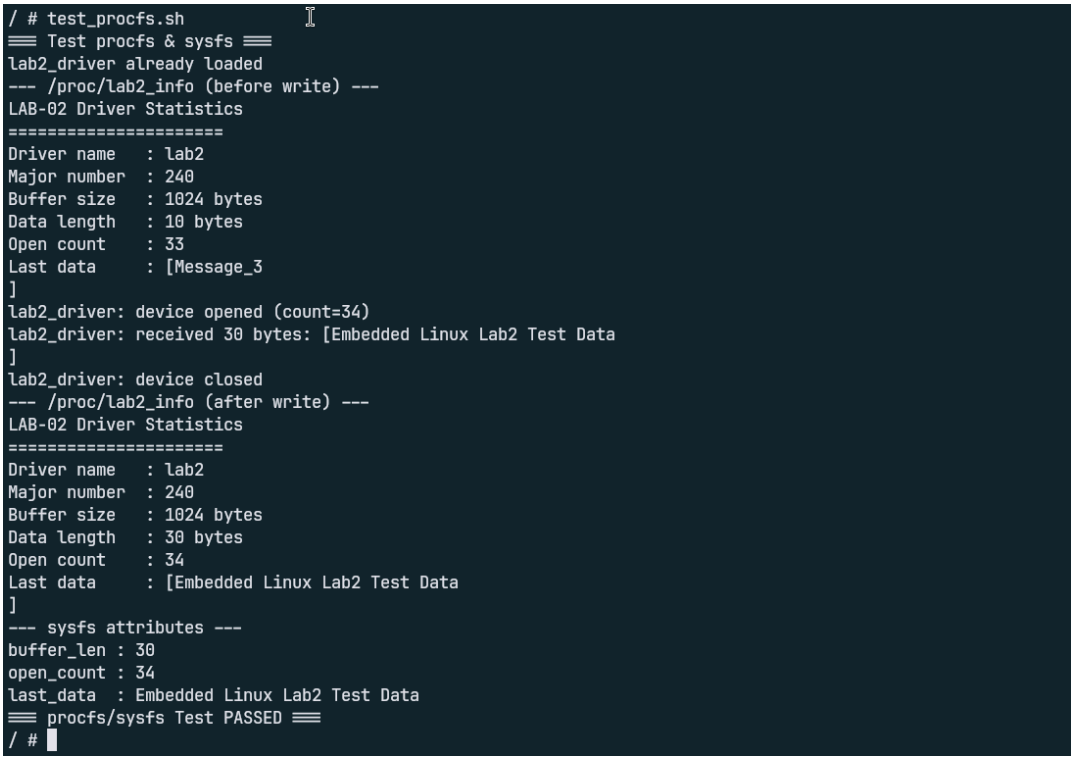


Figure 6: `test_procfs.sh`: `/proc/lab2_info` trước và sau khi ghi, và các thuộc tính sysfs

Trước khi ghi, `/proc/lab2_info` còn giữ dữ liệu của lần ghi cuối từ test trước (`Message_3`, 10 byte, open count 33). Sau khi ghi `Embedded Linux Lab2 Test Data`, data length thành 30 byte và open count thành 34. Ba file sysfs cho đúng các giá trị đó.

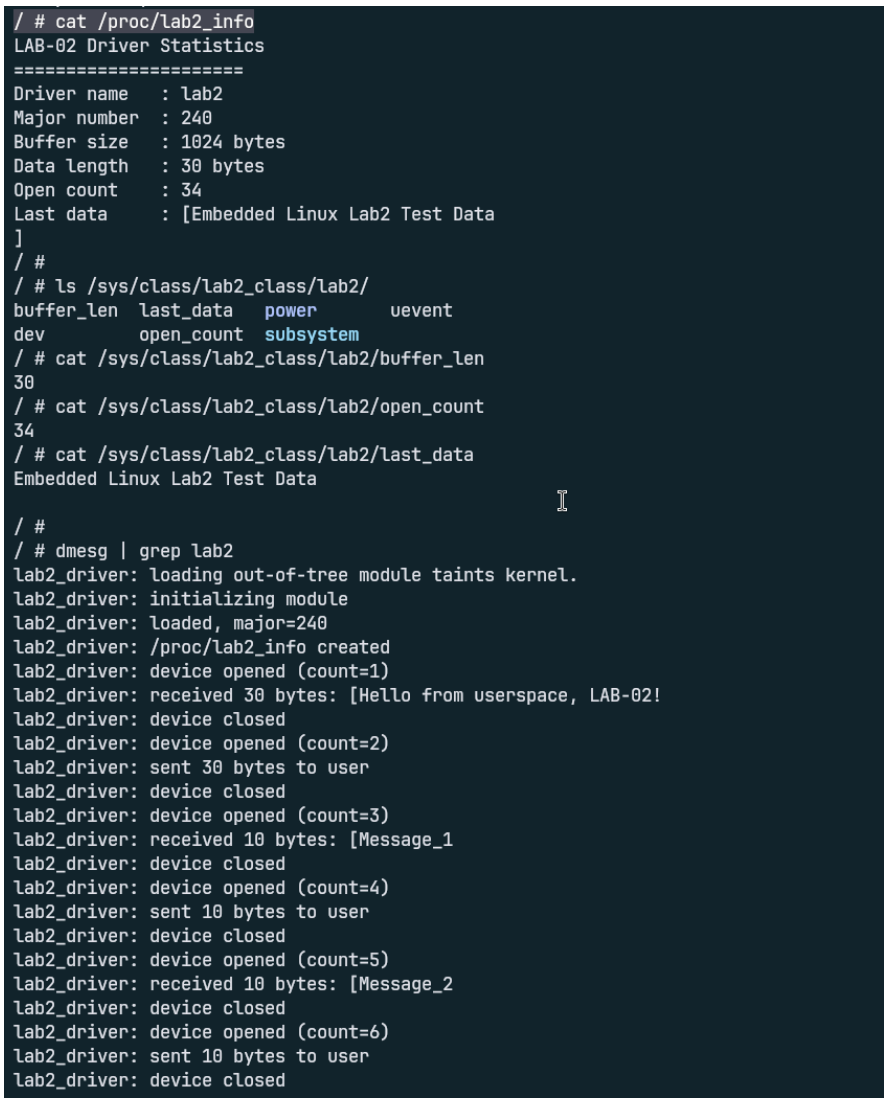


Figure 7: Đọc riêng `/proc/lab2_info`, liệt kê và đọc từng thuộc tính trong `/sys/class/lab2_class/lab2/`

Thư mục sysfs ngoài ba thuộc tính của driver (`buffer_len`, `open_count`, `last_data`) còn có `dev`, `uevent`, `power` và `subsystem` do driver core tự tạo cho mọi device. `dev` chứa `240:0`, đây là thứ mà udev/mdev đọc để tự tạo node `/dev/lab2`.

Phần C – MTD và JFFS2 (chạy trên host)



Figure 8: `mtdinfo /dev/mtd1` và `/proc/mtd`

Sau `modprobe nandsim`, `/proc/mtd` có hai thiết bị. `mtd0` tên `"BIOS"` không phải flash giả lập mà là chip flash SPI chứa firmware thật của laptop, do driver của host tự đưa ra. NAND giả lập là `mtd1`: 32 MiB, erase block 16 KiB (2048 block), trang 512 byte, OOB 16 byte. Vì vậy mọi lệnh trong đề ghi `mtd0` em đều đổi thành `mtd1` (xem mục 6).

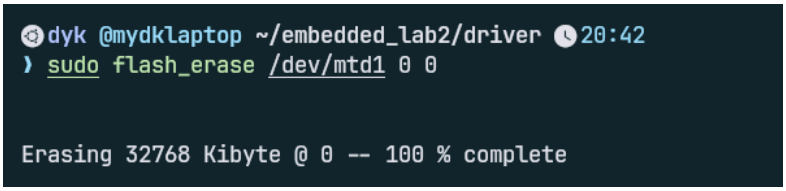


Figure 9: `flash_erase /dev/mtd1 0 0`: xóa toàn bộ 32 MiB

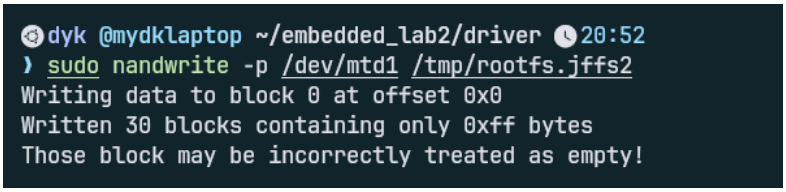


Figure 10: `nandwrite -p /dev/mtd1 /tmp/rootfs.jffs2`

Image JFFS2 được tạo bằng `mkfs.jffs2 --eraseblock=0x4000`, khớp erase block 16 KiB của nandsim. Thông báo "Written 30 blocks containing only 0xff bytes" là do `--pad` đệm image bằng `0xFF`; với JFFS2 các vùng toàn `0xFF` chính là vùng trống đã erase, nên không ảnh hưởng.

```
dyk @mydklaptop ~/embedded_lab2/driver 20:53
❯ sudo mkdir -p /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs2

ls -la /mnt/jffs2/
cat /mnt/jffs2/hello.txt
cat /mnt/jffs2/info.txt
cat /mnt/jffs2/logs/boot.log
total 4
drwxr-xr-x 4 root root    0 Jan  1  1970 .
drwxr-xr-x 3 root root 4096 Sep 28 20:56 ..
-rw-rw-r-- 1 dyk  dyk    18 Sep 28 20:52 hello.txt
-rw-rw-r-- 1 dyk  dyk    20 Sep 28 20:52 info.txt
drwxrwxr-x 2 dyk  dyk     0 Sep 28 20:52 logs
Hello from JFFS2!
Embedded Linux Lab2
Mon Sep 28 08:52:30 PM +07 2026
```

Figure 11: Mount JFFS2 qua `/dev/mtdblock1` và đọc các file

```
dyk @mydklaptop ~/embedded_lab2/driver 21:01
❯ sudo nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd1
hexdump -C /tmp/dump.bin | head -20
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
00000000  85 19 01 e0 31 00 00 00  1d d9 82 42 01 00 00 00  |....1......B....|
00000010  00 00 00 00 02 00 00 00  1e 71 ba 6a 09 08 00 00  |.........q.j....|
00000020  92 1c ba b1 15 da 69 f4  68 65 6c 6c 6f 2e 74 78  |......i.hello.tx|
00000030  74 ff ff ff 85 19 02 e0  56 00 00 00 09 64 e7 62  |t.......V....d.b|
00000040  02 00 00 00 01 00 00 00  b4 81 00 00 e8 03 e8 03  |................|
00000050  12 00 00 00 1e 71 ba 6a  1e 71 ba 6a 1e 71 ba 6a  |.....q.j.q.j.q.j|
00000060  00 00 00 00 12 00 00 00  12 00 00 00 00 00 00 00  |................|
00000070  ee dd 4b b3 d2 39 36 f9  48 65 6c 6c 6f 20 66 72  |..K..96.Hello fr|
00000080  6f 6d 20 4a 46 46 53 32  21 0a ff ff 85 19 01 e0  |om JFFS2!.......|
00000090  30 00 00 00 78 be 3e fa  01 00 00 00 01 00 00 00  |0...x.>.........|
000000a0  03 00 00 00 1e 71 ba 6a  08 08 00 00 09 a6 cb 3c  |.....q.j.......<|
000000b0  a9 de 79 72 69 6e 66 6f  2e 74 78 74 85 19 02 e0  |..yrinfo.txt....|
000000c0  58 00 00 00 3a 13 38 82  03 00 00 00 01 00 00 00  |X...:.8.........|
000000d0  b4 81 00 00 e8 03 e8 03  14 00 00 00 1e 71 ba 6a  |.............q.j|
000000e0  1e 71 ba 6a 1e 71 ba 6a  00 00 00 00 14 00 00 00  |.q.j.q.j........|
000000f0  14 00 00 00 00 00 00 00  8a 74 43 dd 56 6e 9a a4  |.........tC.Vn..|
00000100  45 6d 62 65 64 64 65 64  20 4c 69 6e 75 78 20 4c  |Embedded Linux L|
00000110  61 62 32 0a 85 19 01 e0  2c 00 00 00 5f 56 f1 e0  |ab2.....,..._V..|
00000120  01 00 00 00 02 00 00 00  04 00 00 00 ae 70 ba 6a  |.............p.j|
00000130  04 04 00 00 bb 60 d6 52  40 19 cb d1 6c 6f 67 73  |.....`.R@...logs|

dyk @mydklaptop ~/embedded_lab2/driver 21:04
❯
```

Figure 12: `nanddump` 512 byte đầu của `mtd1`

Dump thô cho thấy JFFS2 lưu dữ liệu dưới dạng các node nối tiếp nhau. Mỗi node bắt đầu bằng magic `85 19` (`0x1985` theo little-endian) và loại node: `01 e0` là node thư mục (dirent) mang tên file `hello.txt`, `02 e0` là node inode mang nội dung `Hello from JFFS2!`. Tiếp theo là cặp node tương tự cho `info.txt` và thư mục `logs`. Không có bảng FAT hay superblock cố định nào; khi mount, JFFS2 quét các node này để dựng lại cây thư mục.

Phần D – BusyBox init tự nạp module



Figure 13: Boot log: rcS nạp `lab2_driver` và tạo `/dev/lab2`

`/etc/inittab` có dòng `::sysinit:/etc/init.d/rcS`, nên rcS chạy trước khi có shell. rcS mount `proc`, `sysfs`, `tmpfs`, `devtmpfs`, kiểm tra file `/lib/modules/5.15.0/lab2_driver.ko` có tồn tại không rồi `insmod`. Log cho thấy đủ chuỗi: kernel báo module out-of-tree làm "taint" kernel (bình thường với module ngoài cây nguồn), driver in `initializing`, `loaded, major=240`, `/proc/lab2_info created`, rồi rcS tạo `/dev/lab2` và in `System initialization complete!`. Output đầy đủ của `test_driver.sh` và `test_procfs.sh` trên hệ đã boot này là các hình ở phần A và B.

Vì module đã được rcS nạp sẵn, bản `test_driver.sh` gốc gọi `insmod` lần nữa sẽ lỗi `File exists`. Em sửa hai script để kiểm tra `lsmod | grep -q '^lab2_driver'` trước, đã nạp thì bỏ qua (dòng `lab2_driver already loaded` trong log).

4. So sánh filesystem

Tiêu chí	ext4	JFFS2	cramfs	UBIFS (trên UBI)
Ghi được	Có	Có	Không (chỉ đọc)	Có
Wear leveling	Không tự làm; dựa vào controller của eMMC/SD	Có, theo kiểu log-structured trong phạm vi một phân vùng	Không cần vì không ghi	Có, do lớp UBI làm trên toàn bộ flash
Nén	Không	Có (zlib, lzo, rtime)	Có (zlib, theo trang)	Có (lzo, zlib, zstd)
Thời gian mount/boot	Nhanh, đọc superblock	Chậm, phải quét toàn bộ node; tăng theo dung lượng	Nhanh	Nhanh, có index trên flash
Chịu mất điện	Journal	Tốt, mỗi node có CRC	Không áp dụng	Tốt, journal + CRC
Thiết bị phù hợp	eMMC, SD, ổ cứng (block device)	Raw NAND/NOR nhỏ	Firmware chỉ đọc, ROM	Raw NAND vừa và lớn

Khuyến nghị cho thiết bị IoT 32 MB NAND cần cập nhật OTA. Em chọn UBI làm lớp quản lý flash, chia thành hai volume rootfs A/B dùng SquashFS (chỉ đọc, nén tốt) và một volume dữ liệu nhỏ dùng UBIFS. Khi cập nhật OTA, thiết bị ghi image mới vào volume đang không chạy, rồi đổi biến môi trường U-Boot để lần boot sau dùng volume đó; nếu boot lỗi thì quay về volume cũ. UBI lo wear leveling và bad block trên toàn bộ chip, nên cả phần ghi thường xuyên (log, cấu hình) lẫn phần ghi lại khi OTA đều được rải đều. JFFS2 vẫn dùng được ở 32 MB và đơn giản hơn, nhưng thời gian mount tăng theo dung lượng và nó không hỗ trợ tốt việc thay cả rootfs một cách an toàn như cách A/B ở trên. ext4 thì không dùng được vì raw NAND không phải block device.

5. Trả lời câu hỏi phân tích

Câu 1. Tại sao không dùng memcpy() giữa kernel buffer và user buffer

Mỗi tiến trình có không gian địa chỉ ảo riêng. Con trỏ `buf` mà `lab2_read()` nhận là một địa chỉ ảo của tiến trình đang gọi (ví dụ `cat`), chỉ có nghĩa thông qua bảng trang của tiến trình đó, và do user tự đưa vào nên không thể tin được. Nó có thể là `NULL`, trỏ vào vùng chưa map, trỏ vào trang đang bị swap ra, hoặc cố ý trỏ vào địa chỉ của kernel.

`memcpy()` không kiểm tra gì cả. Nếu trang chưa có trong RAM, page fault xảy ra trong kernel mà không có chỗ xử lý thì thành kernel oops. Nếu user truyền địa chỉ kernel, driver sẽ vô tình đọc hoặc ghi đè vào bộ nhớ kernel, đây là lỗ hổng bảo mật.

`copy_to_user()`/`copy_from_user()` làm ba việc mà `memcpy()` không làm: kiểm tra cả vùng `[buf, buf+len)` nằm trong phần user của không gian địa chỉ (`access_ok`); thực hiện chép bằng các lệnh được đăng ký trong bảng exception, nên nếu gặp page fault thì kernel hoặc nạp trang vào rồi chép tiếp, hoặc dừng lại và trả về số byte chưa chép được thay vì oops; và trên ARM còn bật/tắt quyền truy cập vùng user (domain) đúng lúc. Trong driver em, giá trị trả về khác 0 được chuyển thành `-EFAULT`, nên một `read()` với con trỏ sai chỉ làm chương trình đó nhận lỗi, kernel không bị ảnh hưởng.

Câu 2. Tại sao cần mutex

`device_buffer` và `buffer_len` là biến toàn cục dùng chung cho mọi tiến trình mở `/dev/lab2`. Trên kernel SMP như của em (4 CPU), hai tiến trình có thể ở trong `lab2_write()` cùng một lúc trên hai CPU khác nhau, và kể cả trên một CPU thì `copy_from_user()` có thể ngủ khi gặp page fault để tiến trình khác chạy chen vào.

Nếu không có mutex: tiến trình A ghi "AAAAAAAAAA" (10 byte), tiến trình B ghi "BBB" (3 byte) cùng lúc. A chép xong 10 byte, B chép đè 3 byte đầu, rồi A đặt `buffer_len = 10` sau B. Kết quả đọc ra là "BBBAAAAAAA", một chuỗi mà không ai ghi. `open_count++` cũng không nguyên tử: đó là đọc–cộng–ghi, hai CPU cùng làm thì có thể mất một lần đếm.

Mutex đảm bảo chỉ một tiến trình được ở trong đoạn chép và cập nhật độ dài tại một thời điểm. Em dùng mutex chứ không dùng spinlock vì `copy_from_user()` có thể ngủ, mà không được ngủ khi đang giữ spinlock.

Khi đọc lại code, em thấy `lab2_read()` và các hàm `*_show()` của sysfs đọc `device_buffer` mà không khóa mutex, nên một lần đọc trùng lúc với lần ghi vẫn có thể lấy được nửa dữ liệu cũ nửa dữ liệu mới. Muốn chặt chẽ thì cần khóa cả ở đường đọc. Trong các test của bài, các lần đọc ghi diễn ra tuần tự nên không gặp trường hợp này.

Câu 3. procfs và sysfs

procfs ban đầu dành cho thông tin về tiến trình (`/proc/<PID>/`), nhưng theo thời gian các driver đổ đủ loại thông tin vào đó, mỗi file một định dạng tự chọn, không có cấu trúc chung. sysfs ra đời cùng driver model: mỗi thư mục là một kobject (device, driver, bus, class), và quy ước là *một file chỉ chứa một giá trị*. Nhờ vậy chương trình và udev đọc được mà không cần phân tích chuỗi, và cây thư mục phản ánh đúng quan hệ giữa các thiết bị. Tài liệu kernel vì thế khuyên không thêm file mới không liên quan tới tiến trình vào `/proc`.

Driver của em có cả hai và thấy rõ khác biệt: `/proc/lab2_info` in một bảng nhiều dòng có nhãn, dễ đọc cho người, viết bằng `seq_file` và `single_open()` (kernel lo phần offset và chia trang, driver chỉ việc `seq_printf`). Còn `/sys/class/lab2_class/lab2/` có ba file `buffer_len`, `open_count`, `last_data`, mỗi file một giá trị, tạo bằng `DEVICE_ATTR_RO` và `sysfs_create_group()` gắn vào kobject của device. Script muốn lấy độ dài buffer chỉ cần `cat buffer_len`, trong khi với procfs phải `grep` rồi cắt chuỗi. Với một driver thật, em sẽ chỉ giữ phần sysfs (hoặc debugfs cho thông tin debug).

Câu 4. Tại sao JFFS2 phải erase cả block trước khi ghi

Trong NAND flash, mỗi bit là một ô nhớ có floating gate. Thao tác ghi (program) chỉ có thể kéo bit từ 1 xuống 0, bằng cách bơm điện tích vào gate. Muốn đưa bit về lại 1 phải rút điện tích ra, và mạch điện của chip chỉ làm được việc này cho cả một khối lớn cùng lúc, gọi là erase block. Ghi tính theo trang (512 byte trên nandsim của em), còn xóa tính theo block (16 KiB, tức 32 trang). Sau khi erase, mọi byte là `0xFF`; đó là lý do `flash_erase` ở phần C đưa cả chip về `0xFF` và vì sao `nandwrite` coi các block toàn `0xFF` là trống.

Vì vậy không thể ghi đè một vài byte tại chỗ. JFFS2 giải quyết bằng cách không bao giờ ghi đè: mỗi thay đổi là một node mới (có số version cao hơn) ghi vào chỗ trống tiếp theo, node cũ bị đánh dấu lỗi thời. Khi hết chỗ, garbage collector chọn một block, chép các node còn hiệu lực sang chỗ khác rồi erase cả block đó. Cách ghi tuần tự này cũng tự rải đều số lần erase lên các block.

ext4 thì làm việc với block device, nơi mỗi sector ghi đè được trực tiếp. Trên eMMC hay SD, chính controller bên trong thẻ (FTL) đã che đi việc erase và wear leveling, nên ext4 không cần biết flash tồn tại. Đặt ext4 lên raw NAND thì không có ai làm việc đó.

Câu 5. insmod trong rcS thất bại thì sao

Hệ thống vẫn boot tiếp. rcS là một shell script bình thường, không có `set -e`, nên một lệnh lỗi chỉ trả về mã khác 0 và script chạy tiếp các dòng sau. BusyBox init cũng không quan tâm rcS thành công hay thất bại: chạy xong mục `sysinit` thì nó chuyển sang `askfirst` và mở shell. Kết quả là người dùng vẫn vào được shell nhưng không có `/dev/lab2`, và mọi thứ phụ thuộc vào driver sẽ lỗi về sau.

rcS của em đã kiểm tra hai trường hợp: file `.ko` không tồn tại thì in `WARNING: lab2_driver.ko not found`, còn `insmod` trả lỗi thì in `ERROR: insmod lab2_driver failed!` và không tạo node. Để chắc chắn hơn, có thể ghi lỗi vào file log trên phân vùng bền vững, thử nạp lại vài lần, hoặc với driver bắt buộc thì cho rcS khởi động lại hệ thống sang bản firmware dự phòng (kết hợp với cơ chế A/B ở mục 4).

6. Nhận xét và khó khăn

Khó khăn 1: `mtd0` trên máy em là flash BIOS thật

Đề bài giả định sau khi `modprobe nandsim` thì NAND giả lập là `/dev/mtd0`, và các lệnh `flash_erase`, `nandwrite`, `nanddump` đều trỏ vào đó. Trên laptop của em, `cat /proc/mtd` cho thấy `mtd0` tên `"BIOS"`, kích thước 32 MiB, erase size 4 KiB: đây là chip SPI chứa firmware của máy mà kernel host đã đưa ra như một thiết bị MTD. Nếu chạy `flash_erase /dev/mtd0 0 0` như đề, firmware sẽ bị xóa và máy không khởi động được nữa. Em kiểm tra tên trong `/proc/mtd` trước, xác định NAND giả lập là `mtd1` (“NAND simulator partition 0”, erase size 16 KiB), và đổi toàn bộ lệnh của phần C sang `mtd1`/`mtdblock1`. Bài học là trước khi chạy lệnh xóa trên thiết bị, luôn kiểm tra tên và kích thước của nó chứ không tin số thứ tự.

Khó khăn 2: `insmod` hai lần

Sau khi thêm phần D, rcS đã nạp module lúc boot, nên `test_driver.sh` và `test_procfs.sh` bản gốc gọi `insmod` lần nữa và lỗi `File exists`. Em sửa hai script để kiểm tra `lsmod` trước, và chỉ `mknod` khi node chưa tồn tại.

Nhận xét

Bài này cho em thấy driver chỉ là một bảng con trỏ hàm mà VFS gọi vào, và mọi thứ còn lại (tạo node, xuất thông tin, nạp lúc boot) đều xoay quanh việc đăng ký bảng đó cho đúng. Phần MTD cho thấy flash thô khác hẳn đĩa: nhìn hexdump mới thấy JFFS2 thực sự chỉ là một chuỗi node nối tiếp, không có cấu trúc cố định nào để ghi đè.